TASK 6: Automated Logging using Triggers & Views

🎯 AIM

To implement an automated audit logging system using SQL triggers to track INSERT and UPDATE operations and create a view to display daily activity reports.

⚙️ ALGORITHM

1. Start
2. Create main table (Employee)
3. Create log table (EmployeeLog)
4. Insert sample employee records
5. Create INSERT trigger to log new records
6. Create UPDATE trigger to log old and new values
7. Perform INSERT and UPDATE operations
8. Store all changes automatically in log table
9. Create VIEW to display today's activities
10. End

Code :

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee Audit Logging System</title>
    <style>
        body {
            font-family: Arial;
            background: #f4f4f4;
            text-align: center;
        }
```

```
        h2 {
            margin-top: 20px;
        }
        table {
            width: 80%;
            margin: 20px auto;
            border-collapse: collapse;
            background: white;
        }
        th, td {
            border: 1px solid #ccc;
            padding: 10px;
            text-align: center;
        }
        th {
            background: #4CAF50;
            color: white;
        }
        .section {
            margin-top: 30px;
        }
        button {
            padding: 10px;
            margin: 10px;
            cursor: pointer;
        }
    </style>
</head>
<body>
```

```
<h2>Employee Audit Logging System (Trigger Simulation)</h2>
<button onclick="updateSalary()">Update Salary (Trigger)</button>
<!-- EMPLOYEE TABLE -->
<div class="section">
<h3>Employee Table</h3>
<table>
    <thead>
        <tr>
            <th>EmpID</th>
            <th>Name</th>
            <th>Department</th>
            <th>Salary</th>
        </tr>
    </thead>
    <tbody id="empTable"></tbody>
</table>
</div>
<!-- LOG TABLE -->
<div class="section">
<h3>Employee Log Table (Auto Trigger)</h3>
<table>
    <thead>
        <tr>
            <th>LogID</th>
            <th>EmpID</th>
            <th>Operation</th>
            <th>Old Salary</th>
            <th>New Salary</th>
        </tr>
```

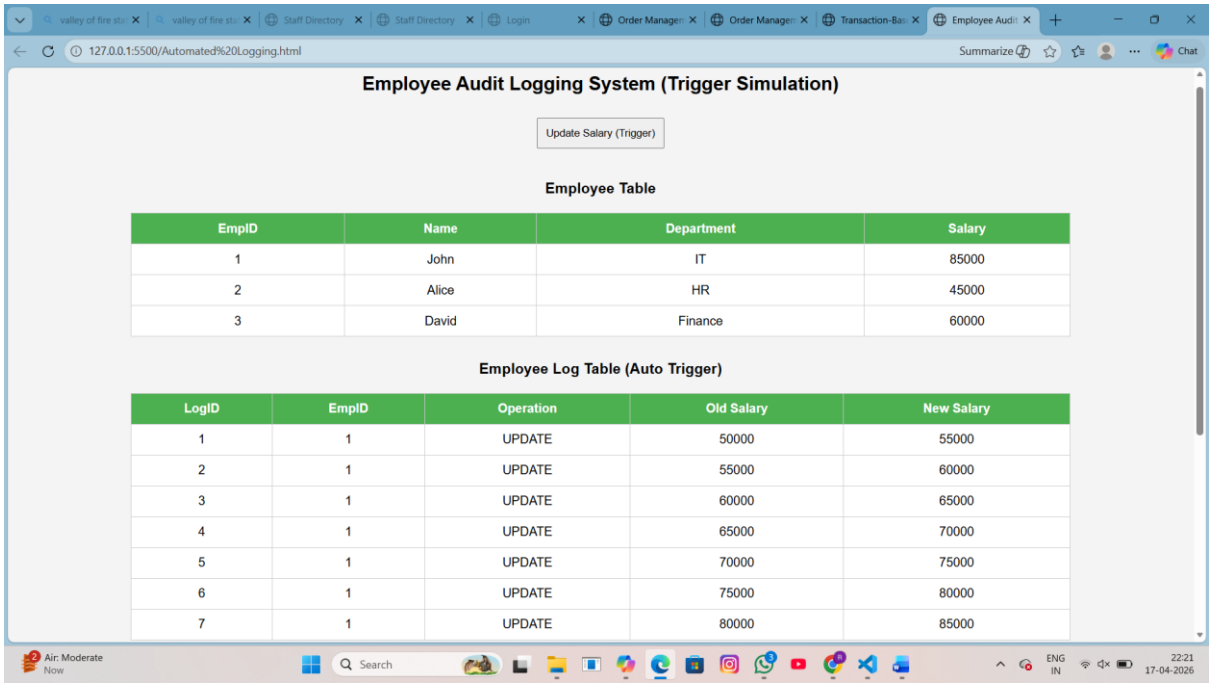
```
    </thead>
    <tbody id="logTable">></tbody>
</table>
</div>
<!-- DAILY VIEW -->
<div class="section">
<h3>Daily Activity Report (VIEW)</h3>
<table>
    <thead>
        <tr>
            <th>EmpID</th>
            <th>Operation</th>
            <th>Old Salary</th>
            <th>New Salary</th>
        </tr>
    </thead>
    <tbody id="viewTable">></tbody>
</table>
</div>
<script>
// EMPLOYEE DATA (like SQL table)
let employees = [
    {id: 1, name: "John", dept: "IT", salary: 50000},
    {id: 2, name: "Alice", dept: "HR", salary: 45000},
    {id: 3, name: "David", dept: "Finance", salary: 60000}];
// LOG TABLE (trigger simulation)
let logs = [];
let logId = 1;
// DISPLAY EMPLOYEES
```

```
function displayEmployees() {
    let table = "";
    employees.forEach(e => {
        table += `
        <tr>
            <td>${e.id}</td>
            <td>${e.name}</td>
            <td>${e.dept}</td>
            <td>${e.salary}</td>
        </tr>`;
    });
    document.getElementById("empTable").innerHTML = table;
}
// TRIGGER SIMULATION (UPDATE SALARY)
function updateSalary() {
    let emp = employees[0]; // update first employee
    let oldSalary = emp.salary;
    let newSalary = oldSalary + 5000;
    // UPDATE EMPLOYEE TABLE
    emp.salary = newSalary;
    // INSERT TRIGGER LOG
    logs.push({
        logId: logId++,
        empId: emp.id,
        operation: "UPDATE",
        oldSalary: oldSalary,
        newSalary: newSalary
    });
    displayEmployees();
```

```
    displayLogs();
    displayView();
}
// DISPLAY LOG TABLE
function displayLogs() {
    let table = "";
    logs.forEach(l => {
        table += `
        <tr>
            <td>${l.logId}</td>
            <td>${l.empId}</td>
            <td>${l.operation}</td>
            <td>${l.oldSalary}</td>
            <td>${l.newSalary}</td>
        </tr>`;
    });
    document.getElementById("logTable").innerHTML = table;
}
// DAILY VIEW (LIKE SQL VIEW)
function displayView() {
    let table = "";
    logs.forEach(l => {
        table += `
        <tr>
            <td>${l.empId}</td>
            <td>${l.operation}</td>
            <td>${l.oldSalary}</td>
            <td>${l.newSalary}</td>
        </tr>`;
```

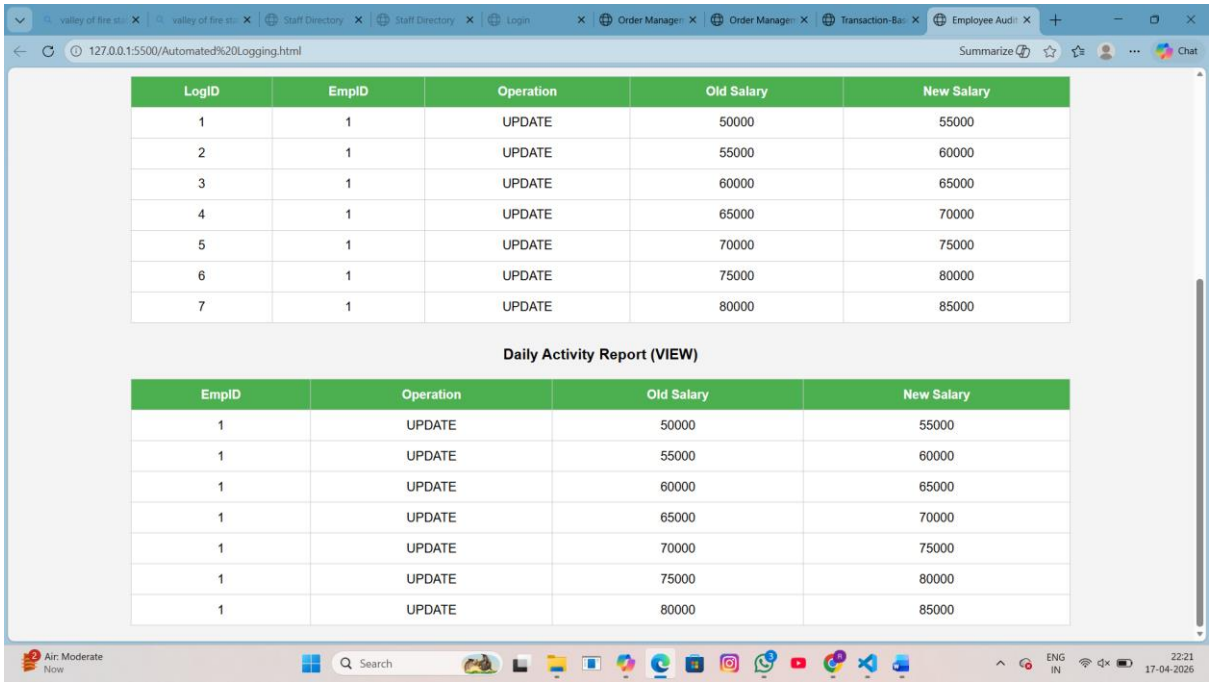
```
    });
    document.getElementById("viewTable").innerHTML = table;
}
// INITIAL LOAD
displayEmployees();
displayLogs();
displayView();
</script>
</body>
</html>
```

Output:



EmpID	Name	Department	Salary
1	John	IT	85000
2	Alice	HR	45000
3	David	Finance	60000

LogID	EmpID	Operation	Old Salary	New Salary
1	1	UPDATE	50000	55000
2	1	UPDATE	55000	60000
3	1	UPDATE	60000	65000
4	1	UPDATE	65000	70000
5	1	UPDATE	70000	75000
6	1	UPDATE	75000	80000
7	1	UPDATE	80000	85000



LogID	EmpID	Operation	Old Salary	New Salary
1	1	UPDATE	50000	55000
2	1	UPDATE	55000	60000
3	1	UPDATE	60000	65000
4	1	UPDATE	65000	70000
5	1	UPDATE	70000	75000
6	1	UPDATE	75000	80000
7	1	UPDATE	80000	85000

Daily Activity Report (VIEW)

EmpID	Operation	Old Salary	New Salary
1	UPDATE	50000	55000
1	UPDATE	55000	60000
1	UPDATE	60000	65000
1	UPDATE	65000	70000
1	UPDATE	70000	75000
1	UPDATE	75000	80000
1	UPDATE	80000	85000

RESULT :

The system successfully simulates SQL triggers and views using HTML and JavaScript. It automatically records employee updates into a log table and displays daily activity reports, demonstrating real-time audit logging functionality used in enterprise database systems.